

PRODUCT REQUIREMENTS DOCUMENT

FAZAI

A comprehensive product requirements document and architecture flow for FAZAI, a personal cash-basis accounting Progressive Web Application designed for simplicity, offline capability, and AI-powered financial management.

Product Requirements Document
& Architecture Flow

June 2026 · v0.4.0

CONFIDENTIAL

Table of Contents

Part I: Product Requirements Document	3
1. Product Overview	3
2. User Personas	3
2.1 Rina - Individual	4
2.2 Budi - Freelancer	4
2.3 Admin Sari - Small Business	4
3. Feature Requirements	5
4. Non-Functional Requirements	7
5. Data Model	8
5.1 User	8
5.2 Account	8
5.3 Transaction	9
5.4 Entry	9
5.5 AccountMonthlySummary	9
5.6 ArchivedTransaction	10
5.7 Default Chart of Accounts	10
6. Business Rules	11
Part II: Architecture Flow	12
7. System Architecture Overview	12
8. Component Architecture	13
9. Data Flow Architecture	14
10. State Management Architecture	15
11. API Architecture	16

12. Security Architecture	16
13. Deployment Architecture	17
14. Technology Stack Decision Record	17

Part I: Product Requirements Document

1. Product Overview

FAZAI is a personal cash-basis accounting Progressive Web Application (PWA) designed for individuals and small business owners who need simple, double-entry bookkeeping without the complexity of enterprise accounting software. The application follows cash-basis accounting principles with full double-entry ledger support, enabling accurate financial tracking through debit-credit journal entries. Every transaction recorded in FAZAI maintains the fundamental accounting equation where total debits equal total credits, ensuring that financial statements are always balanced and audit-ready.

The core philosophy behind FAZAI is captured in the statement: "Accounting should be as simple as tapping your phone." This mobile-first, offline-capable application works without internet connectivity for all core functions, making it ideal for users in areas with unreliable connectivity or those who simply prefer a fast, local-first experience. The application stores all financial data directly in the browser using IndexedDB, eliminating the need for server-side infrastructure and ensuring that sensitive financial information remains on the user's device.

The target users for FAZAI span four primary categories: (1) Individuals tracking personal income and expenses who want a straightforward tool to understand their cash flow; (2) Freelancers managing income from multiple clients who need to track expenses by category and project; (3) Small business owners needing basic bookkeeping capabilities including financial reporting and double-entry ledger management; and (4) Accounting students learning double-entry principles who benefit from a hands-on, interactive learning environment.

FAZAI's key differentiators in the personal accounting space include: an offline-first PWA architecture that ensures core functionality works without network access; an AI-powered transaction assistant that enables natural-language financial queries and transaction creation; comprehensive multi-language support covering English, Bahasa Indonesia, and Chinese; PIN-based quick access for secure yet convenient authentication; and zero server dependency for core features, meaning the application can function entirely on a single device with no recurring infrastructure costs.

2. User Personas

Persona	Role	Tech Level	Primary Need
Rina	Individual	Basic	Track daily spending and savings

Budi	Freelancer	Intermediate	Manage income from multiple clients, track expenses by category
Admin Sari	Small Business	Advanced	Full double-entry bookkeeping, financial reporting, team access

Table 1: User Personas Overview

2.1 Rina - Individual

Rina is a 28-year-old office worker who wants to understand where her money goes each month. She has tried spreadsheet-based tracking but finds it tedious to maintain. Her primary goal is to see a simple dashboard showing her income versus expenses at a glance, without needing to understand accounting terminology. She wants to quickly record a cup of coffee or a grocery run in under 5 seconds. Her pain points include: complex accounting apps that overwhelm her with features she does not need, apps that require internet connectivity (she commutes through areas with poor signal), and the lack of Bahasa Indonesia localization in most financial apps. FAZAI addresses Rina's needs by providing a one-tap expense entry on the dashboard, an offline-first architecture that works during her commute, full Bahasa Indonesia support, and a clean dashboard that shows today's income and expense totals without requiring any accounting knowledge.

2.2 Budi - Freelancer

Budi is a 35-year-old freelance graphic designer who works with 5-8 clients simultaneously. He needs to track income per client and categorize business expenses for tax purposes. His primary goals are to know which clients have paid, how much he has earned this month versus last month, and to generate simple reports for his tax accountant. His pain points include: losing track of which payments correspond to which invoices, mixing personal and business expenses in one account, and spending too much time on bookkeeping when he should be designing. FAZAI helps Budi by allowing him to tag transactions with counterparties, providing monthly Profit and Loss reports for his tax accountant, offering category-based expense tracking that separates business from personal spending, and enabling the AI assistant to answer queries like 'How much did I earn from Client X this month?' without manual report generation.

2.3 Admin Sari - Small Business

Sari is a 42-year-old owner of a small retail shop with 3 employees. She needs proper double-entry bookkeeping to satisfy her accountant and wants to generate financial statements including Balance Sheets and Cash Flow reports. Her primary goals are to maintain accurate financial records that her accountant can audit, control which employees can access sensitive financial data, and export reports in PDF and Excel formats for her bank and tax filings. Her pain points include: enterprise accounting software being too expensive and complex for her needs, difficulty controlling employee access to

financial data, and the inability of simple expense trackers to produce proper financial statements. FAZAI addresses Sari's requirements through its admin/user role system that restricts report access and custom journal entries to administrators, full double-entry ledger with Trial Balance and Balance Sheet reports, PDF and XLSX export capabilities for all financial reports, and a custom journal entry feature that allows her accountant to make adjusting entries directly in the application.

3. Feature Requirements

ID	Feature	Priority	Status	Description
F01	PIN Authentication	Must	Done	6-digit PIN login with admin/user roles
F02	Income Recording	Must	Done	Quick income entry with account selection
F03	Expense Recording	Must	Done	Quick expense entry with category selection
F04	Transaction History	Must	Done	Searchable, filterable transaction list
F05	Dashboard	Must	Done	Balance overview, today's income/expense, recent transactions
F06	AI Assistant	Should	Done	Chat-based transaction creation, financial queries, insights
F07	Trial Balance	Must	Done	Point-in-time debit/credit balance report
F08	Balance Sheet	Must	Done	Assets, liabilities, equity as-of report
F09	Profit & Loss	Must	Done	Period-based income/expense/net profit report
F10	Cash Flow	Must	Done	Period-based inflow/outflow/net change report
F11	Ledger Report	Must	Done	Per-account transaction detail with running balance
F12	PDF Export	Should	Done	Report export to PDF with branded header
F13	XLSX Export	Should	Done	Report export to Excel spreadsheet
F14	Custom Journal Entry	Must	Done	Admin multi-row debit/credit entry
F15	Account Management	Must	Done	CRUD for chart of accounts with toggle
F16	User Management	Must	Done	Admin CRUD for users with PIN/role
F17	Backup & Restore	Must	Done	JSON export/import, factory reset
F18	Opening Balance	Should	Done	Set initial account balances
F19	Multi-language	Should	Done	English, Bahasa Indonesia, Chinese
F20	Dark Mode	Should	Done	System-aware theme switching
F21	PWA Install	Must	Done	Add-to-homescreen capable

ID	Feature	Priority	Status	Description
F22	Offline Core	Must	Partial	Data works offline, but no Service Worker for app shell
F23	Monthly Summaries	Must	Done	Optimized balance calculation via summary table
F24	Data Archive	Should	Done	Auto-archive transactions older than 6 months
F25	AI Account Suggest	Could	Done	Keyword-based account suggestion for entries
F26	AI Delete Action	Should	Done	Delete transactions via AI chat
F27	User Guide	Should	Done	In-app help with accordion sections
F28	Admin Reports Gate	Should	Planned	Restrict report access to admin role
F29	Account Balance Field	Should	Planned	Show current balance on account cards
F30	Custom Txn Form Redesign	Could	Planned	Modern multi-row entry UX overhaul
F31	FAZAI Logo	Must	Planned	Use FAZAI.jpg as app logo
F32	Factory Reset	Must	Done	Full database reset with challenge code

Table 2: Feature Requirements Matrix

Feature prioritization in FAZAI follows the MoSCoW method, categorizing each feature as Must-have, Should-have, or Could-have. Must-have features represent the minimum viable product: without PIN authentication, income/expense recording, transaction history, and the dashboard, the application cannot fulfill its core purpose as an accounting tool. Should-have features enhance the user experience significantly but are not blocking for initial release; these include the AI assistant, PDF/XLSX export, multi-language support, and dark mode. Could-have features are desirable enhancements that would improve the product but can be deferred without impacting core functionality, such as AI account suggestions and the custom transaction form redesign.

The current status reflects a mature MVP: 27 of 32 features are marked as Done, with only 3 features remaining in Planned status and 1 feature (Offline Core) partially implemented. The upcoming roadmap focuses on three key areas. First, completing the PWA offline experience by implementing a Service Worker for the application shell, enabling full offline access including UI loading without network (F22). Second, strengthening the admin role model by gating report access behind admin authentication (F28) and displaying account balances directly on account cards (F29). Third, visual identity and UX improvements including the FAZAI logo integration (F31) and a redesigned custom transaction form that simplifies multi-row debit/credit entry (F30). These planned features target the v0.5.0 release milestone.

4. Non-Functional Requirements

Category	Requirement	Target
Performance	Initial load time	< 3 seconds on 4G
Performance	Report generation	< 2 seconds for 1000 transactions
Performance	AI response time	< 5 seconds
Reliability	Data integrity	Zero data loss on browser crash
Reliability	Offline capability	Core CRUD works without network
Security	PIN authentication	Minimum 6 digits, no plaintext storage
Security	Data encryption	IndexedDB data encrypted at rest
Security	API access	Authenticated endpoints only
Usability	Mobile-first	All screens optimized for 375px width
Usability	Accessibility	WCAG 2.1 Level AA target
Compatibility	Browser support	Chrome, Safari, Firefox, Edge (latest 2 versions)
Compatibility	PWA install	Android Chrome, iOS Safari
Localization	Languages	English, Bahasa Indonesia, Chinese (Simplified)
Maintainability	Test coverage	> 80% on core engine, > 60% overall
Maintainability	TypeScript strict	strict mode enabled, zero any types

Table 3: Non-Functional Requirements

Among the non-functional requirements, three categories demand particular attention due to their impact on user trust and application viability. Performance is critical for user adoption: the initial load target of under 3 seconds on 4G ensures that mobile users, who represent the primary audience, are not deterred by sluggish startup. Report generation under 2 seconds for 1000 transactions is achieved through the pre-calculated monthly summary strategy, which avoids scanning the entire transaction history for each report. Reliability requirements are equally paramount since the application stores financial data locally in IndexedDB. Zero data loss on browser crash is ensured through Dexie.js transaction semantics and the backup/restore feature. Security, while currently minimal with plaintext PIN storage, represents the highest-risk NFR: the planned migration to hashed PINs and encrypted IndexedDB storage is essential before any multi-user production deployment. The current security posture is acceptable only for personal single-device use scenarios.

5. Data Model

The FAZAI data model consists of six core entities stored in IndexedDB via Dexie.js. The model follows a normalized relational structure adapted for client-side document storage. Each entity is described below with its fields, types, and relationships.

5.1 User

Field	Type	Description
id	string (PK)	Auto-generated unique identifier
pin	string	6-digit numeric PIN for authentication
name	string	Display name of the user
role	admin user	Role determining access permissions
isSystem	boolean	Whether this is a system-created user
createdAt	Date	Timestamp of user creation

Table 4: User Entity Fields

5.2 Account

Field	Type	Description
id	string (PK)	Auto-generated unique identifier
code	string	Account code (e.g., 1-1000, 5-1000)
name	string	Account name in English
nameId	string?	Account name in Bahasa Indonesia (optional)
nameZh	string?	Account name in Chinese (optional)
type	asset cashBank liability equity income expense	Account classification type
parentId	string? (FK)	Parent account ID for hierarchical structure
isSystem	boolean	Whether this is a system-created account
isActive	boolean	Whether the account is currently active

createdAt	Date	Timestamp of account creation
-----------	------	-------------------------------

Table 5: Account Entity Fields

5.3 Transaction

Field	Type	Description
id	string (PK)	Auto-generated unique identifier
date	Date	Transaction date
description	string	Human-readable description
counterparty	string	Counterparty name (e.g., client, vendor)
type	income expense custom	Transaction classification
createdBy	string (FK)	Reference to User who created this transaction
createdAt	Date	Timestamp of creation
entries	Entry[]	Array of debit/credit entry objects

Table 6: Transaction Entity Fields

5.4 Entry

Field	Type	Description
id	string (PK)	Auto-generated unique identifier
accountId	string (FK)	Reference to Account
debit	number	Debit amount (0 if credit entry)
credit	number	Credit amount (0 if debit entry)

Table 7: Entry Entity Fields

5.5 AccountMonthlySummary

Field	Type	Description
id	string (PK)	Auto-generated unique identifier
accountId	string (FK)	Reference to Account
year	number	Calendar year (e.g., 2026)
month	number (0-11)	Month index (0=January, 11=December)
totalDebit	number	Sum of all debits for this account/month
totalCredit	number	Sum of all credits for this account/month
lastCalculated	Date	Timestamp of last calculation

Table 8: AccountMonthlySummary Entity Fields

5.6 ArchivedTransaction

Field	Type	Description
(all Transaction fields)	various	Inherits all fields from Transaction entity
archivedAt	Date	Timestamp when the transaction was archived

Table 9: ArchivedTransaction Entity Fields

5.7 Default Chart of Accounts

FAZAI ships with a pre-configured chart of accounts following a standard numerical coding system. The default structure provides a comprehensive starting point for personal and small business accounting:

Code	Account Name	Type
1-0000	Assets	asset
1-1000	Cash & Bank	cashBank
1-1100	Cash on Hand	cashBank
1-1200	Bank Account	cashBank
2-0000	Liabilities	liability

3-0000	Equity	equity
3-1000	Opening Balance	equity
4-0000	Income	income
4-1000	Salary	income
4-2000	Freelance	income
4-3000	Sales	income
4-4000	Interest Income	income
4-9000	Other Income	income
5-0000	Expenses	expense
5-1000	Food & Beverages	expense
5-2000	Transportation	expense
5-9000	Other Expense	expense

Table 10: Default Chart of Accounts

6. Business Rules

Double-Entry Rule: Every transaction in FAZAI must have balanced debits and credits. The sum of all debit amounts must equal the sum of all credit amounts for each transaction. This is enforced at the application layer before any transaction is persisted to IndexedDB. For income and expense quick-entry transactions, the system automatically creates the balancing entry against the appropriate Cash/Bank account.

Cash-Basis Recognition: FAZAI follows cash-basis accounting principles where income is recognized when received (cash inflow) and expenses are recognized when paid (cash outflow). This contrasts with accrual accounting, which recognizes transactions when they are earned or incurred regardless of cash movement. The cash-basis approach simplifies the accounting model for FAZAI's target users who typically do not manage accounts receivable or payable.

Account Type Normal Balances: Asset and CashBank accounts are debit-normal, meaning a debit entry increases their balance and a credit decreases it. Income accounts are credit-normal (credit increases balance). Expense accounts are debit-normal (debit increases balance). Liability and Equity accounts are credit-normal (credit increases balance). This classification determines how the ledger engine calculates running balances and how the Trial Balance, Balance Sheet, and Profit and Loss reports aggregate figures.

Monthly Summary Strategy: Past months use pre-calculated `AccountMonthlySummary` records for fast report generation, while the current month uses live transaction scanning (Month-to-Date). This hybrid approach balances accuracy for the current period with query performance for historical data. The summary rollover process runs at application startup.

Archive Policy: Transactions older than 6 months are automatically archived at startup, moving them from the active `Transaction` table to the `ArchivedTransaction` table. Archived transactions are excluded from routine queries but remain accessible for historical reports. This policy keeps the active dataset small and performant.

Opening Balances: Initial account balances are created as custom transactions against the Opening Balance equity account (3-1000). This ensures that the double-entry equation remains balanced from the very first transaction and that opening balances appear correctly on the Balance Sheet.

PIN Rules and Role Separation: PINs are currently 6-digit numeric codes with no complexity requirements (a minimum complexity check is planned for a future release). The admin role has access to custom journal entries, account and user management, and all reports. The user role can only record income/expense transactions and view the history and dashboard. This separation ensures that only authorized personnel can modify the chart of accounts or create adjusting entries.

Part II: Architecture Flow

7. System Architecture Overview

FAZAI follows a client-heavy, single-page application architecture where the browser serves as both the presentation layer and the data persistence layer. Unlike traditional web applications that rely on a backend server for data storage and business logic, FAZAI executes virtually all operations client-side, with the server playing only a minimal role in AI feature processing.

The entire application is a single Next.js page (`page.tsx`) that uses Zustand state to simulate client-side routing. The Next.js App Router is effectively bypassed: there are no dynamic routes, no server components, and no API middleware beyond the three AI endpoints. Navigation between screens (Dashboard, History, Reports, Settings, Admin Panel) is controlled entirely by the app-store's `currentPage` state variable, which determines which component subtree renders. This architectural decision simplifies deployment and eliminates the need for server-side rendering, but it means that the URL never changes during navigation and browser back/forward buttons do not work as expected.

Data persistence is entirely client-side via Dexie.js, a thin wrapper around the browser's IndexedDB API. There is no server-side database: the Prisma schema and `db.ts` file present in the project are unused

remnants from the initial project scaffold. All financial data, user credentials, and application state reside in the browser's IndexedDB, which persists across page reloads and browser restarts but is lost if the user clears browser data.

The only server-side functionality consists of three API routes for AI features: `/api/ai/chat` (LLM conversation endpoint), `/api/ai/context` (data formatting for LLM context), and `/api/ai/suggest` (account keyword matching for transaction suggestions). These routes use the `z-ai-web-dev-sdk` to communicate with a cloud-based large language model. This architecture was chosen for its simplicity, zero infrastructure cost, and offline capability. The trade-offs are significant: no multi-device sync, no server-side data validation, limited security, and complete data loss if the user clears their browser storage without a backup.

8. Component Architecture

Component	Type	Responsibility	Key Dependencies
PinLogin	Page	Authentication screen	fazai-db, auth-store
Dashboard	Page	Balance overview, quick actions	ledger-engine, auth-store
TransactionForm	Page	Income/expense entry	ledger-engine, AI suggest API
History	Page	Transaction search/list	fazai-db
Settings	Page	User preferences	auth-store, fazai-db
Reports	Page	Report type selector	app-store
ReportViewer	Page	Report display/export	ledger-engine, jsPDF, xlsx
AdminPanel	Container	Admin tab navigation	app-store
AdminUsers	Admin	User CRUD	fazai-db
AdminAccounts	Admin	Account CRUD	fazai-db, ledger-engine
AdminCustomEntry	Admin	Custom journal entry	ledger-engine, fazai-db
AdminBackup	Admin	Data export/import/reset	fazai-db
AdminSettings	Admin	Admin preferences	auth-store, fazai-db
AiChat	Overlay	AI assistant panel	z-ai-web-dev-sdk, fazai-db
BottomNav	Navigation	Tab navigation bar	auth-store, app-store

UserGuide	Overlay	Help documentation	i18n
-----------	---------	--------------------	------

Table 11: Component Architecture

Component communication in FAZAI follows a hybrid pattern combining Zustand global stores with local component state. The auth-store provides authentication context (user ID, role, language) to all components that need it, while the app-store controls navigation by exposing the `currentPage` and `navigate()` function. Components do not communicate directly with each other; instead, they read from shared stores and emit state changes that trigger re-renders in dependent components.

Data loading follows a `useRef`-based load-once pattern: each component maintains a `loadRef` flag that prevents duplicate database queries on re-render. When a component mounts, it checks if `loadRef.current` is false, queries Dexie for its required data, sets the local state, and flips the flag to true. This pattern is simple but has a notable limitation: there is no shared data cache, meaning six or more components independently load the accounts table, causing redundant IndexedDB reads. A future optimization would involve creating a shared data cache layer in Zustand that all components subscribe to.

9. Data Flow Architecture

User Input Flow: When a user performs an action such as recording a transaction, the flow follows this path: User action triggers a component event handler, which calls the ledger-engine module for business logic validation (e.g., double-entry balance check). The ledger-engine then persists the transaction via Dexie.js to IndexedDB. After the write completes, the component re-queries the database to refresh its local state, causing a re-render with the updated data.

Report Generation Flow: When a user selects a reporting period, the `getPeriodDates()` utility computes the start and end dates for the requested period. The ledger-engine then queries the `AccountMonthlySummary` table for completed months and performs a live transaction scan for the current month (MTD). These two result sets are merged to produce the final report data, which the `ReportViewer` component renders as an HTML table. Optionally, the user can export the report to PDF via jsPDF or to XLSX via the `xlsx` library, both of which operate entirely client-side.

AI Chat Flow: When a user sends a message to the AI assistant, the client first gathers financial context from Dexie, including account balances, recent transactions, and monthly summaries. This context is formatted by the `/api/ai/context` endpoint and sent along with the user's message to `/api/ai/chat`. The LLM processes the request and returns a response that may contain an action directive (create transaction, delete transaction, or query result). The client parses the response, executes any actions locally against Dexie via the ledger-engine, and displays the result to the user. The entire flow is stateless from the server's perspective: the server has no knowledge of the user's financial data beyond what is sent in each request.

Startup Flow: When the application loads, Dexie opens the IndexedDB database and the seedDatabase() function checks whether the database is empty. If it is, the default chart of accounts and admin user are created. Next, runStartupMaintenance() executes two critical operations: (1) the monthly summary rollover, which finalizes AccountMonthlySummary records for any months that have elapsed since the last startup, and (2) the archive process, which moves transactions older than 6 months to the ArchivedTransaction table. After maintenance completes, the PIN login screen is displayed.

Monthly Summary Strategy: The hybrid approach to monthly reporting is central to FAZAI's performance. Past months use pre-calculated AccountMonthlySummary records that store totalDebit and totalCredit per account per month. These summaries are calculated once when the month ends and are never recalculated unless the user restores a backup. The current month, however, uses live transaction scanning to compute Month-to-Date (MTD) figures. This ensures that the current month's reports always reflect the latest data without requiring a summary recalculation after every transaction.

10. State Management Architecture

auth-store (Zustand): The authentication store manages the following state: isAuthenticated (boolean), userId (string), userName (string), userRole (admin | user), and lang (en | id | zh). Its actions include login(userId, pin) which validates credentials against Dexie, logout() which clears the auth state, and setLang(lang) which persists the language preference to localStorage for cross-session consistency. The auth-store is the single source of truth for who is currently using the application.

app-store (Zustand): The application store controls client-side routing with the following state: currentPage (string identifying the active screen), previousPage (string for back navigation), reportType (string identifying the selected report type), and selectedTransactionId (string for drill-down navigation). Its primary action, navigate(page), updates currentPage and shifts the current page to previousPage. The setReportType(type) action is called when a user selects a report from the Reports screen before navigating to the ReportViewer.

Local State (useState): Each component manages its own data independently using React's useState hook. There is no global data cache: each component queries Dexie directly when it mounts. This means that the accounts table, for example, is loaded separately by the TransactionForm, Dashboard, AdminAccounts, AdminCustomEntry, and ReportViewer components. While this approach is simple and avoids cache invalidation complexity, it results in redundant IndexedDB reads and potential inconsistencies if one component modifies data that another component has already loaded.

Data Loading Pattern: The useRef-based load-once pattern prevents duplicate database queries on component re-render. Each component declares a loadRef = useRef(false) and checks it in a useEffect before querying Dexie. After the data is loaded and local state is set, loadRef.current is set to true. This pattern works correctly but does not handle cases where data changes in another component (e.g., a new account added in AdminAccounts is not reflected in TransactionForm until the page is reloaded). A

shared data cache in Zustand would resolve this limitation.

11. API Architecture

Endpoint	Method	Purpose	Auth	Input	Output
/api/ai/chat	POST	LLM chat completion	None	{ messages, context, action }	{ content, action }
/api/ai/context	POST	Format financial context	None	{ accounts, transactions, summaries, dashboard }	{ context }
/api/ai/suggest	POST	Account suggestion	None	{ keyword, accounts }	{ suggestions }

Table 12: API Endpoints

The API architecture of FAZAI reflects its client-heavy design philosophy. Unlike traditional three-tier architectures where the server is the data authority, FAZAI's API endpoints serve as thin processing layers that transform client-provided data rather than query a server-side database. Financial data is sent from the client to the API, not fetched from the server. This means the /api/ai/chat endpoint receives the user's financial context (accounts, transactions, summaries) as part of the request payload, formats it for the LLM, and returns the model's response without storing any data server-side.

The /api/ai/suggest endpoint performs simple keyword matching against the account list to suggest accounts for transaction entry (e.g., typing 'coffee' suggests 'Food & Beverages'). This could theoretically be done client-side, but the current implementation routes it through an API for consistency with the LLM interaction pattern. The /api/ai/context endpoint formats raw financial data into a structured context string suitable for LLM consumption, handling number formatting, currency symbols, and date localization. None of the three endpoints require authentication, which is a known security gap documented in Section 12.

12. Security Architecture

Current State: FAZAI's security model is minimal, reflecting its design as a personal, single-device application. The 6-digit PIN is stored in plaintext within IndexedDB, meaning that anyone with access to the browser's developer tools can read all user credentials. Authentication state is stored in localStorage with no session tokens or expiration. API endpoints have no authentication whatsoever: any client that can reach the server can invoke the AI chat, context formatting, and suggestion endpoints. There is no

CSRF protection, no rate limiting on PIN attempts, and no encryption of IndexedDB data at rest.

Planned Improvements: The security roadmap includes several critical enhancements. First, PIN hashing with bcrypt or argon2 to prevent credential exposure even if IndexedDB is compromised. Second, session token generation upon successful PIN validation, with configurable expiration and refresh mechanics. Third, API authentication middleware that validates session tokens on every request. Fourth, rate limiting on PIN attempts (e.g., 5 attempts per minute) to prevent brute-force attacks. Fifth, encryption of sensitive IndexedDB data at rest using the Web Cryptography API, with the encryption key derived from the user's PIN.

Risk Assessment: The current security posture is suitable for personal single-device use where the primary threat model is casual snooping rather than targeted attack. The PIN provides a basic access control mechanism that prevents incidental access by other household members. However, FAZAI in its current state is not suitable for shared devices or multi-user production environments without significant security hardening. The absence of server-side validation means that a compromised client could inject arbitrary transactions, and the lack of audit logging makes it impossible to detect unauthorized modifications after the fact.

13. Deployment Architecture

FAZAI is deployed behind a Caddy reverse proxy listening on port 81, which forwards requests to the Next.js application running on port 3000. The public domain is fazai.space-z.ai, with Caddy handling TLS termination and automatic HTTPS certificate management. The Next.js application is built using next build with standalone output mode and runs on the Bun runtime for improved startup performance.

The PWA configuration includes a manifest.json with standalone display mode, a theme color matching the application's accent color, and icon definitions for various screen densities. This enables the add-to-homescreen functionality on both Android Chrome and iOS Safari. Currently, there is no CI/CD pipeline configured: deployments are performed manually by running next build and restarting the application process. There is no containerization (Docker) and no monitoring or error tracking infrastructure. Application health is monitored only through manual verification and the Caddy access logs.

14. Technology Stack Decision Record

Decision	Choice	Rationale	Trade-off
Framework	Next.js 16	PWA support, React ecosystem	Overkill for SPA - no SSR used
Database	Dexie/IndexedDB	Offline-first, zero infra cost	No server sync, browser-dependent

State	Zustand	Lightweight, simple API	No DevTools like Redux
UI	shadcn/ui + Radix	Accessible primitives, customizable	Large dependency count
AI	z-ai-web-dev-sdk	Integrated LLM access	Vendor lock-in
Auth	Custom PIN	Simple, offline-capable	No real security
Export	jsPDF + xlsx	Client-side generation	xlsx has CVE concerns
Build	Turbopack	Fast dev server	New, some edge cases

Table 13: Technology Stack Decision Record

Each technology decision in FAZAI was driven by the application's core requirements of simplicity, offline capability, and zero infrastructure cost. The most significant trade-off is the choice of Next.js as the framework: while it provides excellent PWA support and the React ecosystem, FAZAI does not use server-side rendering, API routes for data, or dynamic routing, making Next.js functionally equivalent to a Create React App setup with additional complexity. The Dexie/IndexedDB choice enables true offline operation but introduces data fragility, as browser data clearing results in permanent data loss without a backup. The xlsx library for Excel export carries known CVE vulnerabilities, and a migration to a safer alternative such as ExcelJS is recommended for production use.